MODULE 20

Deep Learning Project with Simple RNN

Lectures Covered

Lecture 1 — Getting Started with RNN Project
Lecture 2 — Traditional Embedding Generation Techniques
Lecture 3 — Embedding Layer for Capturing Context Info
Lecture 4 — RNN Project Initial Steps
Lecture 5 — Training Simple RNN without Embedding Layer
Lecture 6 — Training with Embedding Layer in Simple RNN
Lecture 7 — Creating Streamlit UI for the App
Lecture 8 — Pushing Larger Files to GitHub and Deployment

Lecture 1: Getting Started with RNN Project

End-to-end next-word prediction app using Simple RNN on Shakespeare's text. Covers two model variants (with/without embedding), a Streamlit UI, and cloud deployment.

Project Structure & Setup

```
rnn-next-word/
├── data/shakespeare.txt      # corpus
├── models/
│   ├── rnn_no_embed.h5
│   └── rnn_with_embed.h5
├── tokenizer.pkl
├── app.py                    # Streamlit app
└── requirements.txt

pip install tensorflow numpy scikit-learn streamlit
```

Load & Inspect Corpus

```
with open('data/shakespeare.txt', 'r') as f:
    text = f.read()

words = text.lower().split()
vocab = set(words)
print(f'Words: {len(words):,}  |  Vocab: {len(vocab):,}')
```

Lecture 2: Traditional Embedding Generation Techniques

Before learned embeddings, words were converted to numbers using hand-crafted methods. Each has serious limitations for sequential tasks.

Technique	Description | Key Limitation
One-Hot Encoding	Binary vector of size |V|, one 1 at word index | No semantics; explodes with vocab size
Bag of Words	Vector of word counts in a document | Destroys word order entirely
TF-IDF	Count weighted by rarity across docs | Still bag-of-words; no sequential info
Word2Vec (static)	Dense 300-dim vectors; 'king'-'man'+'woman'='queen' | Fixed; not task-specific

```
# One-Hot (baseline used in Lecture 5)
vocab = ['the','cat','sat','on','mat']
w2i   = {w:i for i,w in enumerate(vocab)}
vec   = [0]*len(vocab); vec[w2i['cat']] = 1  # [0,1,0,0,0]

# Word2Vec similarity
from gensim.models import KeyedVectors
wv = KeyedVectors.load_word2vec_format('GoogleNews-vectors-negative300.bin', binary=True)
print(wv.most_similar(positive=['king','woman'], negative=['man'], topn=1))
# [('queen', 0.71)]
```

Why all of these fall short for RNNs

One-hot / BoW / TF-IDF: no semantic similarity — 'cat' and 'kitten' are orthogonal.
Word2Vec: static — vectors don't adapt to your specific task or domain.

Solution: a trainable Embedding layer learned end-to-end with the RNN.

Lecture 3: Embedding Layer for Capturing Context Info

An Embedding layer is a trainable lookup table of shape (vocab_size, embedding_dim). Each word index maps to a dense vector — learned jointly with the RNN to minimise task loss.

```
from tensorflow.keras.layers import Embedding
import tensorflow as tf

# Input:  integer indices  shape = (batch, seq_len)
# Output: dense vectors    shape = (batch, seq_len, embed_dim)
embed = Embedding(input_dim=10000, output_dim=64, input_length=50)

x = tf.constant([[1, 42, 99, 7]])   # 4 word indices
print(embed(x).shape)               # (1, 4, 64)

# Use pre-trained GloVe weights (optional)
embed_layer = Embedding(vocab_size, 100,
                        weights=[embedding_matrix],
                        trainable=False)  # freeze = feature extraction
```

Setting	trainable=False | trainable=True
Weights updated?	No — GloVe/W2V locked | Yes — adapts to your task
Best use case	Small dataset, similar domain | Large dataset or different domain
Training speed	Faster (fewer params) | Slower

Key advantage over one-hot

One-hot RNN input dim = vocab_size (e.g. 10,000). Dense input dim = 64.
Semantically similar words cluster together — RNN generalises across them.
Embeddings are task-optimised; Word2Vec is general-purpose.

Lecture 4: RNN Project — Initial Steps

Transform raw text into structured (X, y) training pairs through a 4-step preprocessing pipeline used by both model variants.

```
import re, numpy as np, pickle
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split

# Step 1 — Clean text
text = re.sub(r'[^a-z\s]', '', raw.lower())

# Step 2 — Build tokenizer
tok = Tokenizer()
tok.fit_on_texts([text])
vocab_size  = len(tok.word_index) + 1
token_list  = tok.texts_to_sequences([text])[0]
pickle.dump(tok, open('tokenizer.pkl','wb'))

# Step 3 — Sliding window sequences (SEQ_LEN=10)
SEQ_LEN = 10
seqs = [token_list[i-SEQ_LEN:i+1] for i in range(SEQ_LEN, len(token_list))]
seqs = np.array(seqs)       # shape: (N, 11)
X, y = seqs[:,:-1], seqs[:,-1]   # X:(N,10)  y:(N,)
```

```
# Step 4 — One-hot labels + split
y_cat = to_categorical(y, num_classes=vocab_size)
X_train,X_val,y_train,y_val = train_test_split(X,y_cat,test_size=0.2)
```

Lecture 5: Training Simple RNN without Embedding Layer

Baseline model: convert integer indices to one-hot on-the-fly via a Lambda layer, then pass high-dimensional sparse vectors through the RNN.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense, Lambda
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
import tensorflow as tf

model = Sequential([
    Lambda(lambda x: tf.one_hot(tf.cast(x,tf.int32), vocab_size),
           input_shape=(SEQ_LEN,)),          # (batch,10) -> (batch,10,V)
    SimpleRNN(128, activation='tanh'),        # many-to-one
    Dense(64, activation='relu'),
    Dense(vocab_size, activation='softmax'),
])

model.compile(optimizer='adam', loss='categorical_crossentropy',
              metrics=['accuracy'])

model.fit(X_train, y_train, epochs=50, batch_size=128,
    validation_data=(X_val, y_val),
    callbacks=[EarlyStopping(patience=5, restore_best_weights=True),
               ModelCheckpoint('models/rnn_no_embed.h5', save_best_only=True)])

# Inference
def predict_next(model, tok, seed, top_k=3):
    tokens = tok.texts_to_sequences([seed])[0][-SEQ_LEN:]
    tokens = [0]*(SEQ_LEN-len(tokens)) + tokens
    probs  = model.predict(np.array([tokens]), verbose=0)[0]
    top    = probs.argsort()[-top_k:][::-1]
    return [(tok.index_word.get(i,'?'), round(float(probs[i]),4)) for i in top]
```

Limitation of this approach

RNN input size = vocab_size (could be 5,000–50,000) — very high-dimensional sparse input.
No semantic structure: 'cat' and 'kitten' are completely unrelated to the model.
Use this as a baseline to compare against the embedding version in Lecture 6.

Lecture 6: Training with Embedding Layer in Simple RNN

Replace the Lambda one-hot with a trainable Embedding layer. The RNN now receives dense 64-dim vectors — fewer parameters, better semantics, higher accuracy.

```
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense

model_emb = Sequential([
    Embedding(vocab_size, 64, input_length=SEQ_LEN),  # (batch,10)->(batch,10,64)
    SimpleRNN(128, activation='tanh'),
    Dense(64, activation='relu'),
    Dense(vocab_size, activation='softmax'),
])

model_emb.compile(optimizer='adam', loss='categorical_crossentropy',
                  metrics=['accuracy'])
```

```
model_emb.fit(X_train, y_train, epochs=100, batch_size=256,
    validation_data=(X_val, y_val),
    callbacks=[EarlyStopping(patience=5, restore_best_weights=True),
               ModelCheckpoint('models/rnn_with_embed.h5', save_best_only=True)])

# Inspect learned embeddings
from sklearn.metrics.pairwise import cosine_similarity
E = model_emb.layers[0].get_weights()[0]   # shape: (vocab_size, 64)
idx = tok.word_index['love']
sims = cosine_similarity(E[idx:idx+1], E)[0].argsort()[-6:][::-1][1:]
print([tok.index_word[i] for i in sims])   # semantically similar words
```

Metric	No Embedding | With Embedding
RNN input dim	vocab_size (~5,000+) | 64
Total params	~5M (vocab-dependent) | ~1M
Training speed	Slower | Faster
Val accuracy	Baseline | Typically +5–15%
Semantic info	None | Learned task-specific

Lecture 7: Creating Streamlit UI for the App

Streamlit turns a Python script into an interactive web app with no HTML/CSS needed. We build a side-by-side comparison of both models.

```
# app.py
import streamlit as st, numpy as np, pickle
from tensorflow.keras.models import load_model

st.set_page_config(page_title='Next Word Predictor', page_icon='brain')

@st.cache_resource          # load once, stay in memory
def load_assets():
    tok   = pickle.load(open('tokenizer.pkl','rb'))
    m1    = load_model('models/rnn_no_embed.h5')
    m2    = load_model('models/rnn_with_embed.h5')
    return tok, m1, m2

tok, m_no, m_emb = load_assets()
SEQ_LEN = 10

def predict(model, seed, k=5):
    tokens = tok.texts_to_sequences([seed])[0][-SEQ_LEN:]
    tokens = [0]*(SEQ_LEN-len(tokens)) + tokens
    probs  = model.predict(np.array([tokens]), verbose=0)[0]
    top    = probs.argsort()[-k:][::-1]
    return [(tok.index_word.get(i,'?'), float(probs[i])) for i in top]

st.title('Next Word Prediction — Simple RNN')
seed  = st.text_input('Enter phrase:', 'to be or not to be that is the')
top_k = st.slider('Predictions', 1, 10, 5)

if st.button('Predict', type='primary'):
    col1, col2 = st.columns(2)
    with col1:
        st.subheader('Without Embedding')
        for word, prob in predict(m_no, seed, top_k):
            st.progress(prob, text=f'{word}  {prob:.2%}')
    with col2:
        st.subheader('With Embedding')
        for word, prob in predict(m_emb, seed, top_k):
```

```
            st.progress(prob, text=f'{word}  {prob:.2%}')

# Run with:  streamlit run app.py
```

Lecture 8: Pushing Larger Files to GitHub and Deployment

Keras .h5 model files often exceed GitHub's 100 MB per-file limit. Git LFS (Large File Storage) stores the actual bytes on a separate server and keeps only a pointer in the repo.

Git LFS Setup

```
# Install Git LFS
brew install git-lfs          # Mac
sudo apt-get install git-lfs  # Ubuntu

git lfs install
git lfs track '*.h5'          # track model files
git lfs track '*.pkl'

git add .gitattributes
git add models/ tokenizer.pkl app.py requirements.txt
git commit -m 'Add models and Streamlit app'
git push origin main
```

Alternative — Download Model at Runtime

```
# Store model on Google Drive; download in app.py on cold start
import gdown, os

@st.cache_resource
def load_assets():
    if not os.path.exists('models/rnn_with_embed.h5'):
        os.makedirs('models', exist_ok=True)
        gdown.download('https://drive.google.com/uc?id=YOUR_ID',
                       'models/rnn_with_embed.h5')
    return load_model('models/rnn_with_embed.h5')
```

Deploy to Streamlit Cloud

```
# 1. Push to GitHub (with Git LFS for .h5 files)
# 2. Go to share.streamlit.io -> Connect GitHub
# 3. Select repo, branch, app.py -> Deploy
# 4. App live at: https://your-username-appname.streamlit.app
```

Checklist Item	Notes
requirements.txt	All packages with pinned versions
.gitattributes	Committed before model files — *.h5 tracked by LFS
Model paths	Use relative paths — absolute paths break on server
@st.cache_resource	Prevents model reload on every user interaction
tokenizer.pkl	Must be in repo — required for inference

End of Module 20 Notes — Generative AI Course